Лабораторна робота 7

Формальна специфікація сповіщень про бронювання

Z-нотація та VDM-SL

Виконав

Некрасов Артур Дмитрович
Група ІПЗ-23-3

2026

1 Фрагмент системи та правила

Формалізовано життєвий цикл сповіщення, яке створюється після успішного бронювання примірника.

Мета роботи — описати один стан і ті самі переходи в Z та VDM-SL, вивести передумову надсилання й перевірити обробку відмов. Критичні випадки — повторний запуск під час активної спроби, надсилання до retryAt і відповідь від уже завершеної спроби.

У моделі один запис про бронювання має одне сповіщення. Операції змінюють стан сповіщення атомарно. Помилка надсилання залишає бронювання збереженим.

Код	Бізнес-правило	Підстава
П1	Кожному збереженому в цьому фрагменті бронюванню відповідає одне сповіщення. Один ідентифікатор сповіщення належить лише одному бронюванню.	Лаб. №2, Reservation ↔ Notification 1 : 1
П2	Успішне збереження реєструє нову пару. Сповіщення починає з Queued і нуля спроб.	UC03, атомарне збереження A09 Лаб. №4; T01 Лаб. №5
П3	Підготовка переводить Queued у Ready лише після успішної перевірки даних.	T02 [validPayload] Лаб. №5; A03 Лаб. №6
П4	Пауза дозволена з Queued або Ready. Відновлення повертає до тієї самої фази.	T04–T06 Лаб. №5, мілка історія H
П5	Спробу можна почати з Ready або з RetryWait, коли поточний час досяг retryAt.	T03, T11 Лаб. №5; A05 і C01 Лаб. №6
П6	Старт збільшує лічильник на один, очищає час повтору та переводить у WaitingResult. Друга спроба під час очікування заборонена.	entry WaitingResult Лаб. №5; C02–C04 Лаб. №6
П7	Результат змінює стан лише в WaitingResult і лише для поточного номера спроби. Підтвердження шлюзу дає Sent; застарілий результат ігнорується.	T07–T09, T12 Лаб. №5; C05–C07 Лаб. №6
П8	Відмова або тайм-аут задає retryAt = now + 5 хв. У RetryWait можна встановити новий час, строго пізніший за now.	T08, T10 Лаб. №5; C06 Лаб. №6
П9	Знімок стану повертає фазу, бізнес-статус, лічильник і час повтору без зміни стану. Час повтору є лише в RetryWait.	Notification у Лаб. №2 і №5; A02, A04, A06 Лаб. №6
П10	Невдала спроба викликати Send повертає причину: невідоме сповіщення, неготова фаза або ще рано. Стан зберігається.	Уточнення C01r і гілок результату сценарію A Лаб. №6

2 Неоднозначності вимог

Рішення нижче задають межі контрактів. Вони потрібні, щоб однаковий вхід в однаковому стані давав однаковий результат у двох нотаціях.

Питання	Прийняте рішення	Наслідок для моделі
H1 Що означає успішний send()?	Started означає, що створено спробу. Статус SENT означає підтвердження шлюзу; прочитання сповіщення не відстежується.	Старт дає WaitingResult. Тільки поточний позитивний RecordResult дає Sent.
H2 Чи можна повторити точно в retryAt?	Можна з першої хвилини, коли now ≥ retryAt. now і at відлічуються від спільного початку в цілих хвилинах.	Виклик у 104 для retryAt = 105 повертає TooEarly; у 105 — Started.
H3 Чи є 3 спроби загальним лімітом?	Три спроби — бюджет одного запуску NotificationWorker у Лаб. №6. Лічильник Notification є наскрізним і не обнуляється між запусками.	attempts має тип $\mathbb{N}$ / nat. Лічильник localTries належить сценарію A і не входить у стан.
H4 Що відновлює resume()?	Відновлюється фаза перед паузою: Queued або Ready. Пауза завжди створює запис історії.	Кожен Paused має resumeTo. Перехід H → Queued за замовчуванням недосяжний із Paused; новий запис починає з Queued.
H5 Що робити з повторною або пізньою відповіддю?	Під час WaitingResult застосовується лише відповідь з поточним номером. Після виходу з фази запізнілі події відсіює маршрутизатор спроб.	Відмінний номер у WaitingResult залишає стан незмінним. Після Sent передумова RecordResult хибна.

Межі стану

dom noticeOf містить бронювання, збережені в UC03. Операція Register відображає успішний коміт цієї транзакції. У проєкції зберігається історичний зв’язок і після скасування. Обробка UC07, термін дії бронювання, стан примірника, текст повідомлення та мережевий транспорт лишаються поза фрагментом.

now надає довірений монотонний годинник сервісу. Для одного сповіщення зміни серіалізовано, а шлюз передає ідентифікатор і додатний номер спроби. Тайм-аут нормалізується в RecordResult з ok = false. У фрагменті Send моделює зміну стану при запуску, а взаємодія зі шлюзом і тимчасовим DeliveryAttempt лишається в сценарії C.

Фази Queued і Ready розгортають складений стан підготовки з Лаб. №5. Sent охоплює негайне завершення T12; запит Snapshot дозволений і після завершення. Внутрішні ідентифікатори записано латиницею для однозначного читання коду в інструментах.

3 Специфікація Z

Типи і константи

```
[RESERVATION, NOTICE]

PHASE ::= Queued | Ready | Paused | WaitingResult | RetryWait | Sent
MESSAGE ::= PENDING | RETRY | SENT
BOOL ::= no | yes
SENDREPORT ::= Started | UnknownNotice | NotReady | TooEarly

PREP == {Queued, Ready}
TIME == ℕ
ATTEMPT == ℕ₁
```

Аксіоматичне визначення

```
RETRY_DELAY : TIME
statusOf : PHASE → MESSAGE
```

```
RETRY_DELAY = 5
statusOf = {Queued ↦ PENDING, Ready ↦ PENDING, Paused ↦ PENDING,
            WaitingResult ↦ PENDING, RetryWait ↦ RETRY, Sent ↦ SENT}
```

RESERVATION і NOTICE — непрозорі ідентифікатори. PHASE задає шість листових фаз, а statusOf відображає їх у три бізнес-статуси. RETRY_DELAY має значення 5 хвилин.

Стан та ініціалізація

DeliveryState

```
noticeOf : RESERVATION ⇸ NOTICE
phase    : NOTICE ⇸ PHASE
attempts : NOTICE ⇸ ℕ
retryAt  : NOTICE ⇸ TIME
resumeTo : NOTICE ⇸ PREP
```

```
dom noticeOf ∈ 𝔽 RESERVATION
dom phase = ran noticeOf
dom attempts = dom phase
dom retryAt = {n : dom phase | phase n = RetryWait}
dom resumeTo = {n : dom phase | phase n = Paused}
∀ n : dom phase | phase n ∈ {WaitingResult, RetryWait, Sent} •
   attempts n > 0
∀ n : dom phase | phase n ∈ {Queued, Ready, Paused} • attempts n = 0
```

Init

```
DeliveryState'
```

```
noticeOf' = ∅ ∧ phase' = ∅ ∧ attempts' = ∅
retryAt' = ∅ ∧ resumeTo' = ∅
```

Рядки предикатів читаються як кон’юнкція та нумеруються I1–I7 згори вниз. I2 зв’язує дві сутності: область phase дорівнює множині сповіщень, на які посилаються бронювання. I3

гарантує наявність лічильника. I4 та I5 задають області часу повтору й історії; I6 та I7 розрізняють фази до та після першої спроби.

Правило П1 виражене структурою noticeOf. Часткова функція дозволяє одне значення для ключа, а ін’єктивність забороняє спільне сповіщення двом бронюванням. Типи retryAt і resumeTo дають не більше одного часу й однієї попередньої фази для кожного n. Init задає порожню проєкцію.

Реєстрація та підготовка

ΔDeliveryState включає стан до та після переходу. Штрих позначає значення після операції, ? — вхід, ! — вихід. Незмінні змінні вказано рівностями рамки.

```
┌─ Register
──────────────────────────────────────────────────────────────────────────────
┌──────────┐
│ ΔDeliveryState
│ r? : RESERVATION; n? : NOTICE
├──────────────────────────────────────────────────────────────────────────────
└──────────┘
┌──────────────────────────────────────────────────────────────────────────────
│ r? ∉ dom noticeOf ∧ n? ∉ ran noticeOf
│ noticeOf' = noticeOf ∪ {r? ↦ n?}
│ phase' = phase ∪ {n? ↦ Queued}
│ attempts' = attempts ∪ {n? ↦ 0}
│ retryAt' = retryAt ∧ resumeTo' = resumeTo
└──────────────────────────────────────────────────────────────────────────────
```

Register додає лише свіжі r? і n?. Запис Queued та лічильник 0 з’являються разом зі зв’язком. Це формальний наслідок успішного коміту UC03.

```
┌─ Prepare
──────────────────────────────────────────────────────────────────────────────
┌──────────┐
│ ΔDeliveryState
│ n? : NOTICE; valid? : BOOL
├──────────────────────────────────────────────────────────────────────────────
└──────────┘
┌──────────────────────────────────────────────────────────────────────────────
│ n? ∈ {n : dom phase | phase n = Queued} ∧ valid? = yes
│ phase' = phase ⊕ {n? ↦ Ready}
│ noticeOf' = noticeOf ∧ attempts' = attempts
│ retryAt' = retryAt ∧ resumeTo' = resumeTo
└──────────────────────────────────────────────────────────────────────────────
```

Prepare приймає результат валідації valid? як вхід. Без підтверджених даних перехід T02 незастосовний.

```
┌─ Pause
──────────────────────────────────────────────────────────────────────────────
┌──────────┐
│ ΔDeliveryState
│ n? : NOTICE
├──────────────────────────────────────────────────────────────────────────────
└──────────┘
┌──────────────────────────────────────────────────────────────────────────────
│ n? ∈ {n : dom phase | phase n ∈ PREP}
│ phase' = phase ⊕ {n? ↦ Paused}
│ resumeTo' = resumeTo ∪ {n? ↦ phase n?}
│ noticeOf' = noticeOf ∧ attempts' = attempts ∧ retryAt' = retryAt
└──────────────────────────────────────────────────────────────────────────────
```

Resume

$$
\begin{array}{l}
\Delta \text{DeliveryState} \\
n? : \text{NOTICE} \\
\hline
n? \in \operatorname{dom} \text{resumeTo} \\
\text{phase}' = \text{phase} \oplus \{n? \mapsto \text{resumeTo}\ n?\} \\
\text{resumeTo}' = \{n?\} \lhd \text{resumeTo} \\
\text{noticeOf}' = \text{noticeOf} \land \text{attempts}' = \text{attempts} \land \text{retryAt}' = \text{retryAt}
\end{array}
$$

Pause зберігає поточний підстан у resumeTo; Resume бере його звідти й вилучає запис історії. Інваріант I5 робить guard Resume еквівалентним фазі Paused.

У формулах $\oplus$ позначає перевизначення відображення, $\lhd$ — вилучення ключів, $\lhd$ — обмеження області, $\mathscr{F}$ — множину скінченних підмножин.

Надсилання і результат спроби

SendEnabled

DeliveryState
n? : NOTICE; now? : TIME

n? ∈ {n : dom phase | phase n = Ready}
  ∨ n? ∈ {n : dom retryAt | retryAt n ≤ now?}

Send

ΔDeliveryState
SendEnabled

phase' = phase ⊕ {n? ↦ WaitingResult}
attempts' = attempts ⊕ {n? ↦ (attempts n? + 1)}
retryAt' = {n?} ⩤ retryAt
noticeOf' = noticeOf ∧ resumeTo' = resumeTo

SendEnabled допускає Ready та RetryWait з досягнутим часом. У Send індекс спроби зростає один раз, стан переходить у WaitingResult, час повтору вилучається.

ResultInput

DeliveryState
n? : NOTICE; a? : ATTEMPT; ok? : BOOL; now? : TIME

n? ∈ {n : dom phase | phase n = WaitingResult}

RecordOk

ΔDeliveryState
ResultInput

a? = attempts n? ∧ ok? = yes
phase' = phase ⊕ {n? ↦ Sent}
noticeOf' = noticeOf ∧ attempts' = attempts
retryAt' = retryAt ∧ resumeTo' = resumeTo

RecordFail

ΔDeliveryState
ResultInput

a? = attempts n? ∧ ok? = no
phase' = phase ⊕ {n? ↦ RetryWait}
retryAt' = retryAt ∪ {n? ↦ (now? + RETRY_DELAY)}
noticeOf' = noticeOf ∧ attempts' = attempts ∧ resumeTo' = resumeTo

StaleResult

ΞDeliveryState
ResultInput

a? ≠ attempts n?

RecordResult ≜ RecordOk ∨ RecordFail ∨ StaleResult

ResultInput обмежує запис результату фазою WaitingResult. RecordOk і RecordFail розрізняють підтвердження та відмову поточної спроби. StaleResult з Ξ залишає стан незмінним незалежно від ok?, коли номер не збігається.

Повна операція та запит

Reschedule

ΔDeliveryState
n? : NOTICE; at? : TIME; now? : TIME

n? ∈ dom retryAt ∧ now? < at?
retryAt' = retryAt ⊕ {n? ↦ at?}
noticeOf' = noticeOf ∧ phase' = phase
attempts' = attempts ∧ resumeTo' = resumeTo

Snapshot

ΞDeliveryState
n? : NOTICE
p! : PHASE; status! : MESSAGE; count! : ℕ; due! : 𝒻TIME

n? ∈ dom phase
p! = phase n? ∧ status! = statusOf p! ∧ count! = attempts n?
due! = ran ({n?} ◁ retryAt)

Reschedule змінює тільки час, зберігаючи його область визначення. Snapshot не змінює жодної змінної: due! порожня або одноелементна множина.

UnknownError

ΞDeliveryState
n? : NOTICE; now? : TIME; report! : SENDREPORT

n? ∉ dom phase ∧ report! = UnknownNotice

NotReadyError

ΞDeliveryState
n? : NOTICE; now? : TIME; report! : SENDREPORT

n? ∈ {n : dom phase | phase n ∉ {Ready, RetryWait}}
report! = NotReady

TooEarlyError

ΞDeliveryState
n? : NOTICE; now? : TIME; report! : SENDREPORT

n? ∈ {n : dom retryAt | now? < retryAt n}
report! = TooEarly

Success ≜ [report! : SENDREPORT | report! = Started]

SendTotal ≜ (Send ∧ Success) ∨ UnknownError ∨ NotReadyError ∨ TooEarlyError

Успіх і три помилкові схеми мають однакові входи й результат. Started означає запуск спроби, а Ξ у кожній помилковій гілці зберігає всі п’ять змінних стану.

4 Специфікація VDM-SL

Модуль LibraryNotice використовує окремі типи записів для ідентифікаторів. Додатний номер спроби задає інваріант типу AttemptNo, а п’ять полів DeliveryState відповідають схемі Z.

```
types
  ReservationId :: value : token;
  NoticeId :: value : token;
  Minute = nat;
  AttemptNo = nat
  inv a == a > 0;

  PrepPhase = <Queued> | <Ready>;
  Phase = PrepPhase | <Paused> | <WaitingResult> | <RetryWait> | <Sent>;
  MessageState = <PENDING> | <RETRY> | <SENT>;
  SendReport = <Started> | <UnknownNotice> | <NotReady> | <TooEarly>;
  SnapshotView ::
    step   : Phase
    status : MessageState
    count  : nat
    due    : set of Minute;

values
  RETRY_DELAY : Minute = 5;

state DeliveryState of
  noticeOf : inmap ReservationId to NoticeId
  phase    : map NoticeId to Phase
  attempts : map NoticeId to nat
  retryAt  : map NoticeId to Minute
  resumeTo : map NoticeId to PrepPhase
inv mk_DeliveryState(link, p, a, d, h) ==
  -- I1: VDM maps are finite. I2-I7 are the following predicates.
  dom p = rng link
  and dom a = dom p
  and dom d = {n | n in set dom p & p(n) = <RetryWait>}
  and dom h = {n | n in set dom p & p(n) = <Paused>}
  and (forall n in set dom p &
    p(n) in set {<WaitingResult>, <RetryWait>, <Sent>} => a(n) > 0)
  and (forall n in set dom p &
    p(n) in set {<Queued>, <Ready>, <Paused>} => a(n) = 0)
init s == s = mk_DeliveryState({|->}, {|->}, {|->}, {|->}, {|->})
end
```

Тип inmap є скінченною частковою ін’єкцією; його скінченність виражає I1. Предикат inv відтворює I2–I7, а init задає п’ять порожніх відображень.

Явна та неявна функції

```
  StatusOf : Phase -> MessageState
  StatusOf(p) ==
    if p = <Sent> then <SENT>
    elseif p = <RetryWait> then <RETRY>
    else <PENDING>;

  RetryDeadline(now : Minute) due : Minute
  post due = now + RETRY_DELAY;
```

Явна StatusOf обчислює статус із фази. Неявна RetryDeadline задає єдиний допустимий результат через post. Після відмови операція обчислює now + RETRY_DELAY і перевіряє контракт через post_RetryDeadline.

Контракти операцій VDM-SL

Send та RecordResult відтворюють однойменні схеми Z. У post позначення x~ означає старе значення, x — нове. Оновлення фаз і залежних полів відбуваються разом через atomic, щоб інваріант виконувався на межах операції.

```
Send : NoticeId * Minute ==> ()
Send(n, now) ==
  atomic (
    phase := phase ++ {n |-> <WaitingResult>};
    attempts := attempts ++ {n |-> attempts(n) + 1};
    retryAt := {n} <-: retryAt
  )
pre SendEnabled(phase, retryAt, n, now)
post phase = phase~ ++ {n |-> <WaitingResult>}
  and attempts = attempts~ ++ {n |-> attempts~(n) + 1}
  and retryAt = {n} <-: retryAt~
  and noticeOf = noticeOf~ and resumeTo = resumeTo~;

RecordResult : NoticeId * AttemptNo * bool * Minute ==> ()
RecordResult(n, a, ok, now) ==
  if a <> attempts(n) then skip
  elseif ok then phase := phase ++ {n |-> <Sent>}
  else
    atomic (
      phase := phase ++ {n |-> <RetryWait>};
      retryAt := retryAt munion {n |-> now + RETRY_DELAY}
    )
pre n in set {m | m in set dom phase & phase(m) = <WaitingResult>}
post noticeOf = noticeOf~ and attempts = attempts~ and resumeTo = resumeTo~
  and (if a <> attempts~(n) then phase = phase~ and retryAt = retryAt~
  elseif ok then phase = phase~ ++ {n |-> <Sent>} and retryAt = retryAt~
  else phase = phase~ ++ {n |-> <RetryWait>}
    and retryAt = retryAt~ munion {n |-> now + RETRY_DELAY}
    and post_RetryDeadline(now, retryAt(n)));

pure Snapshot : NoticeId ==> SnapshotView
Snapshot(n) ==
  return mk_SnapshotView(phase(n), StatusOf(phase(n)), attempts(n),
                         rng ({n} <: retryAt))
pre n in set dom phase
post RESULT = mk_SnapshotView(phase(n), StatusOf(phase(n)), attempts(n),
                              rng ({n} <: retryAt))
```

SendTotal реалізує розбиття на чотири випадки: викликає Send тільки в гілці Started, а три відмови повертає без присвоєнь. Операція pure Snapshot читає стан і повертає результат; у due від нуля до одного значення.

Повний модуль у додатку містить також Register, Prepare, Pause, Resume та Reschedule з pre і post. Для кожної змінної стану, якої не торкається операція, у постумові переходу вказано рівність зі старим значенням.

5 Обов’язки доведення

Розглядаємо довільний стан, що задовольняє I1–I7. Для кожної операції з передумовою треба показати існування стану після переходу, який виконує типи та інваріант.

ОД1 Початковий стан існує

Вибираємо п’ять порожніх відображень. Порожній зв’язок є скінченною ін’єкцією, тому I1 виконується. Області та значення в I2–I5 порожні з обох боків. Квантори I6 та I7 істинні на порожній області. Отже, значення Init задовольняють DeliveryState′ і реалізують init у VDM-SL.

ОД2 Реєстрація зберігає інваріант

У Register новий r відсутній в області noticeOf, а n — серед його значень. Тому додавання пари не створює конфлікту ні ключа, ні значення; додавання одного елемента зберігає скінченність (I1). За I2–I3 n також відсутній в обох основних відображеннях. ran noticeOf, dom phase і dom attempts одночасно зростають на {n}, тому I2–I3 зберігаються.

Queued не належить RetryWait або Paused, отже I4 та I5 виконуються без нових записів у retryAt і resumeTo. attempts′ n = 0 задовольняє I7, а I6 не застосовується до нової фази. Для решти сповіщень усі дані незмінні.

ОД3 Старт спроби зберігає інваріант

SendEnabled дає відомий n у Ready або RetryWait, у другому випадку — з досягнутим часом. I3 гарантує, що attempts n визначено. Send перевизначає наявні ключі phase й attempts, зберігає їхні області та не змінює noticeOf (I1–I3).

Перехід у WaitingResult вилучає n із множини сповіщень у RetryWait, якщо він був у ній. Видалення n з retryAt дає саме нову область I4. Початкова й кінцева фази не Paused, тому I5 не змінюється. attempts′ n = attempts n + 1 > 0 забезпечує I6; n не належить фазам I7 після переходу. Для інших n рамка зберігає всі предикати.

ОД4 Результат спроби зберігає інваріант

У WaitingResult за I4–I5 немає retryAt і resumeTo для цього n, а за I6 attempts n > 0. RecordOk переводить n у Sent і зберігає додатний лічильник, тож I4–I7 виконуються. RecordFail додає єдиний новий запис n ↦ now + 5 у retryAt одночасно з фазою RetryWait. Сума належить TIME, а додавання не конфліктує з ключем. I4 отримує ту саму нову область; I5–I7 збережено.

У двох гілках результату жоден ключ phase та attempts не додається й не вилучається, тому I1–I3 збережено. За a? ≠ attempts n? діє StaleResult, і всі інваріанти зберігаються через Ξ.

Інші зміни стану

Prepare, Pause і Resume змінюють фази лише всередині множини I7, не торкаючись лічильника й часу повтору. Pause додає свіжий ключ до resumeTo з допустимим значенням PREP, Resume його вилучає. Так I5 зберігається в обох переходах. Reschedule підміняє лише наявне значення retryAt, тож область I4 незмінна; at належить TIME. Решта інваріантів зберігаються завдяки рамці.

Передумова і повнота SendTotal

ОД5 Обчислення передумови Send

Передумова схеми операції виходить після приховування всіх компонентів стану після переходу:

$$\text{pre Send} \triangleq \exists\, \text{DeliveryState}' \bullet \text{Send}$$

Будь-який свідок DeliveryState′ для Send має задовольняти включену схему SendEnabled. Навпаки, якщо SendEnabled істинна, рівності Send однозначно задають п’ять нових відображень. ОД3 доводить їхню типізованість і виконання I1–I7. Отже, pre Send ≙ SendEnabled. З I4 отримуємо таку форму умови для відомого n:

$$\text{phase } n = \text{Ready} \lor (\text{phase } n = \text{RetryWait} \land \text{retryAt } n \leq \text{now})$$

Умова	Навіщо потрібна
n відомий, $n \in \text{dom phase}$	I3 забезпечує визначеність attempts n. Без перевірки для невідомого ID значення лічильника не існує; проста вставка фази створила б n без пов’язаного бронювання.
Фаза Ready або RetryWait	Забороняє старт до підготовки та під час поточної спроби. Наприклад, запуск із Queued після збільшення лічильника міг би зберегти I1–I7, але порушив би П3 і П5. Paused також потребував би видалення історії, якого Send не виконує.
retryAt $n \leq$ now у RetryWait	I4 гарантує наявність часу, але не встановлює його відношення до поточного моменту. Без цієї умови перехід у 104 при retryAt = 105 зберіг би інваріант, проте порушив би П5.

Чотири взаємовиключні випадки

Для будь-якого типізованого n і now у допустимому стані: якщо n невідомий, обирається UnknownError. Для відомого n у Ready обирається Send. Для RetryWait I4 гарантує визначений час: now < retryAt дає TooEarlyError, інакше обирається Send. Решта відомих фаз дають NotReadyError.

Таким чином, pre SendTotal істинна на всіх допустимих станах і типізованих входах. У гілці успіху стан існує за ОД3; у кожній відмові свідком є старий стан. На межі now = retryAt лише успішна гілка застосовна.

6 Простежуваність правил

Позначки I1–I7 відповідають рядкам інваріанта Z; VDM-SL зберігає їхній зміст через типи та inv. Посилання T і A/C вказують на переходи Лаб. №5 та повідомлення Лаб. №6.

Код	Z	VDM-SL	Елемент UML
П1	noticeOf : RESERVATION ⇻ NOTICE; I2	inmap ReservationId to NoticeId; dom p = rng link	Лаб. №2: асоціація Reservation ↔ Notification 1 : 1.
П2	Init; Register	init; Register з atomic і перевіркою свіжих ID	UC03 та A09 Лаб. №4: спільний запис; T01 Лаб. №5.
П3	Prepare: Queued і valid? = yes	Prepare: pre validPayload і фаза Queued	Лаб. №5 T02; Лаб. №6 A03 prepare().
П4	Pause, Resume; I5; PREP	PrepPhase; resumeTo; Pause і Resume	Лаб. №5 T04–T06: Preparation, Paused, H.
П5	SendEnabled; pre Send	SendEnabled і pre Send	Лаб. №5 T03/T11; Лаб. №6 A05 awaitDue, C01 send.
П6	Send: attempts′, WaitingResult, вилучення retryAt; I6	Send: atomic; постумова; I6	Лаб. №5 entry WaitingResult; Лаб. №6 C02–C04.
П7	ResultInput; RecordOk, RecordFail, StaleResult	RecordResult: guard WaitingResult, гілки за a і ok	Лаб. №5 T07–T09/T12; Лаб. №6 C05–C07 ідентифікують спробу.
П8	RETRY_DELAY; RecordFail; Reschedule; I4	RetryDeadline; RecordResult(false); Reschedule	Лаб. №5 T08/T10 scheduleRetry; Лаб. №6 C06 тайм-аут.
П9	Snapshot з Ξ; statusOf; I3–I4	pure Snapshot; StatusOf; SnapshotView	Лаб. №2 MessageState і поля Notification; Лаб. №6 A02/A04/A06.
П10	SendTotal і три схеми Ξ	SendTotal: SendReport та незмінність у post	Уточнення C01r (ознака старту) й охоронних умов A/C Лаб. №6.

Відповідність діаграмі класів

Елемент Лаб. №2	Формальний відповідник	Зміст
Reservation.id	RESERVATION; ReservationId; dom noticeOf	Ідентифікатори збережених у проєкції бронювань.
Notification.id	NOTICE; NoticeId; dom phase	Кожен ID має рівно один запис фази й лічильника.
Reservation ↔ Notification	noticeOf, часткова ін’єкція; I2	Зв’язок 1 : 1 для всіх зареєстрованих пар, включно з історією.
Notification.state : MessageState	statusOf(phase n); StatusOf	Бізнес-статуси PENDING, RETRY, SENT виводяться з фази Лаб. №5.
Notification.attempts : Integer	attempts : NOTICE ⇸ ℕ; map NoticeId to nat	Нуль до першої спроби, додатне значення після старту.
Notification.retryAt [0..1]	retryAt : NOTICE ⇸ TIME; map NoticeId to Minute	Немає значення поза RetryWait; I4 уточнює множинність 0..1.
Notification.send() : Boolean	Send і SendTotal	Boolean true відповідає Started. Уточнений результат містить код відмови.
Notification.scheduleRetry(at)	Reschedule(n, at, now)	Зміна часу лише у RetryWait. Параметр now дає годинник.

Розбіжності та пропозиції

P1. Три значення MessageState в Лаб. №2 не відрізняють готове повідомлення від активної спроби: обидва PENDING. Рекомендація — додати явну фазу з Лаб. №5 і позначити /state як похідний бізнес-статус.

P2. У Лаб. №2 немає даних для відновлення H. Уточнення resumeTo [0..1] або еквівалентний стан підготовки має зберігатися разом з ознакою паузи; інакше неможливо відрізнити повернення до Queued від повернення до Ready.

P3. Boolean send() і асинхронна відповідь шлюзу описують різні події. У контракті класу слід позначити, що true означає запуск. SendReport можна повернути з сервісної обгортки, а recordResult(a, ok) та SnapshotView позначити як уточнення з Лаб. №5–6.

DeliveryAttempt із сценарію C представляє тимчасовий таймер і маршрутизацію відповіді. У стані збережено тільки фазу і номер спроби; створення, пробудження worker та знищення тимчасового об’єкта не змінюють зв’язок Reservation ↔ Notification.

7 Схеми у редакторі

На скріншотах показано копії підготовлених специфікацій, відкриті в Mousepad. Текст схем та операцій доступний у додатках.

File Edit Search View Document Help

```
RETRY_DELAY = 5
statusOf = {Queued ↦ PENDING, Ready ↦ PENDING, Paused ↦ PENDING,
            WaitingResult ↦ PENDING, RetryWait ↦ RETRY, Sent ↦ SENT}
```

```
DeliveryState
noticeOf : RESERVATION ⇸ NOTICE
phase    : NOTICE ⇸ PHASE
attempts : NOTICE ⇸ ℕ
retryAt  : NOTICE ⇸ TIME
resumeTo : NOTICE ⇸ PREP
```

```
dom noticeOf ∈ 𝔽 RESERVATION
dom phase = ran noticeOf
dom attempts = dom phase
dom retryAt = {n : dom phase | phase n = RetryWait}
dom resumeTo = {n : dom phase | phase n = Paused}
∀ n : dom phase | phase n ∈ {WaitingResult, RetryWait, Sent} •
    attempts n > 0
∀ n : dom phase | phase n ∈ {Queued, Ready, Paused} • attempts n = 0
```

```
Init
DeliveryState'
```

```
noticeOf' = ∅ ∧ phase' = ∅ ∧ attempts' = ∅
retryAt' = ∅ ∧ resumeTo' = ∅
```

```
Register
```

Рисунок 1 — DeliveryState, інваріанти та Init у Z

File Edit Search View Document Help

Lab7_Z.txt × Lab7_VDM.vdmsl ×

```
    resumeTo := {n} <-: resumeTo
  )
pre n in set dom resumeTo
post phase = phase~ ++ {n |-> resumeTo~(n)}
  and resumeTo = {n} <-: resumeTo~
  and noticeOf = noticeOf~ and attempts = attempts~ and retryAt = retryAt~;

Send : NoticeId * Minute ==> ()
Send(n, now) ==
  atomic (
    phase := phase ++ {n |-> <WaitingResult>};
    attempts := attempts ++ {n |-> attempts(n) + 1};
    retryAt := {n} <-: retryAt
  )
pre SendEnabled(phase, retryAt, n, now)
post phase = phase~ ++ {n |-> <WaitingResult>}
  and attempts = attempts~ ++ {n |-> attempts~(n) + 1}
  and retryAt = {n} <-: retryAt~
  and noticeOf = noticeOf~ and resumeTo = resumeTo~;

SendTotal : NoticeId * Minute ==> SendReport
SendTotal(n, now) ==
  if n not in set dom phase then return <UnknownNotice>
  elseif SendEnabled(phase, retryAt, n, now) then
    (Send(n, now); return <Started>)
  elseif phase(n) = <RetryWait> then return <TooEarly>
  else return <NotReady>
```

Рисунок 2 — Send із pre та post і початок SendTotal у VDM-SL

Перевірка синтаксису і сценаріїв

VDMJ 4.7.0-P (збірка 260414) у строгому режимі розібрав і перевірив типи LibraryNotice без помилок та попереджень. В інтерпретаторі з увімкненими перевірками pre, post та inv пройшло 17 із 17 контрольних перевірок.

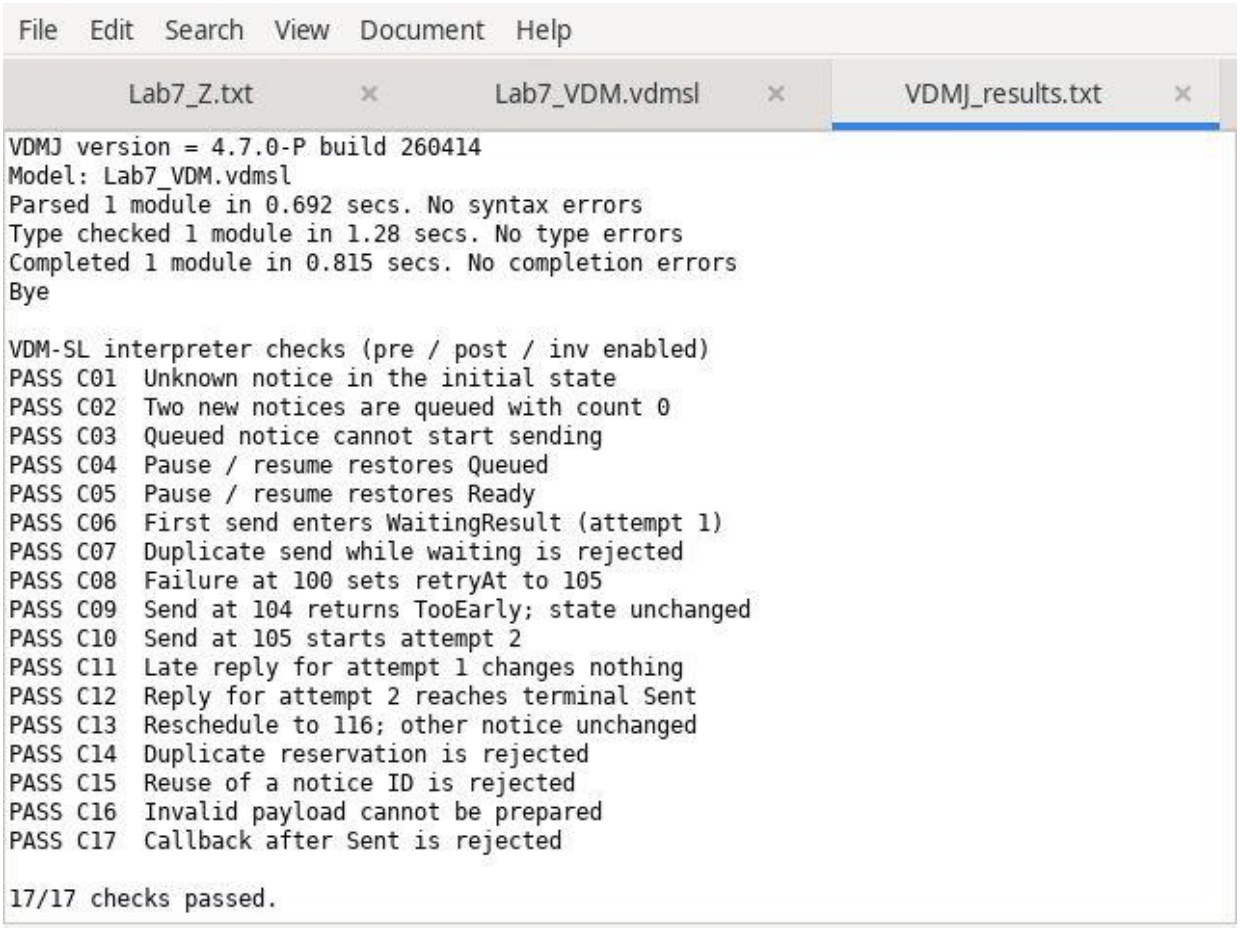


Рисунок 3 — Фактичний журнал перевірки VDMJ, відкритий у Mousepad

Перевірки	Що підтверджено
C01–C05	Невідомий ID, ініціалізація двох записів, відмова в Queued, обидва повернення з Paused.
C06–C10	Старт, відмова під час активної спроби, retryAt = 105, TooEarly у 104 і старт у 105.
C11–C13	Застаріла відповідь без змін, поточний успіх, перенесення часу на 116 і незалежність другого запису.
C14–C17	VDMJ відхиляє повторні ID, невалідні дані та callback після Sent за відповідними pre.

Відтворення: розпакувати файли й запустити python3 Lab7_verify.py шлях_до_vdmj.jar. Скрипт запускає перевірку моделі та окремі сценарії в Lab7Checks.vdmsl. Неформальні докази розділу 5 стосуються довільних допустимих станів; прогін перевіряє конкретні траси.

8 Протокол використання ШІ

Запит до ШІ: переглянути умову та зразки, формалізувати фрагмент університетської бібліотеки у двох нотаціях, зіставити з UML, перевірити специфікацію та підготувати звіт. Перед остаточним оформленням зіставлено передумови, рамки і зміни полів та перевірено контрприклади.

Знайдена проблема або ризик	Перевірка і виправлення
У зразку повна операція обмежена відомими ідентифікаторами.	Для SendTotal потрібна повнота на всіх типізованих ID. Додано UnknownError; C01 підтвердив гілку, а ОД5 покриває всі випадки.
Чернетка тестового модуля зверталася до назв LibraryNotice без кваліфікації.	VDMJ не розпізнав типи та виклики. Вказано модуль у назвах; перевірку двох модулів повторено.
Чернетка тестів вкладала виклики Snapshot у булеві вирази.	Строгий режим VDMJ видав попередження. Знімки спочатку записуються в локальні змінні, а порівнюються як значення.
Ліміт трьох спроб можна було помилково перенести у глобальний інваріант.	Порівняно з loop сценарію A. Лічильник спроб залишається наскрізним; бюджет одного виклику контролює worker.

9 Висновки

Одна модель описує зв’язок бронювання зі сповіщенням та узгоджує фази підготовки, очікування і повтору. Відокремлено факт запуску від підтвердження шлюзу, визначено точну межу повтору і правила ігнорування застарілих відповідей. Виведено передумову Send, обґрунтовано початковий стан і збереження інваріантів для реєстрації, старту та запису результату.

Формалізація показала, що для UML потрібно явно відрізняти бізнес-статус від фази, зберігати історію паузи та уточнити контракт send(). VDM-SL модуль пройшов перевірку типів і всі 17 сценарійних перевірок.